

Main chapter ko concept-by-concept unpack karunga, saath hi examples ka actual behavior aur chapter draft mein jo incomplete ya misleading points hain unhe clearly mark karunga. # Chapter 2: How Objects Work, Detailed Hinglish Explanation

Is chapter ka main purpose ye samjhana hai ki JavaScript object sirf key-value pairs ka container nahi hota. Har object aur uski properties ke peeche kuch internal rules hote hain jo decide karte hain ki property read, update, delete, copy, enumerate ya inherit hogi ya nahi.

Important: Ye chapter abhi **Work in Progress** hai. Sealed, Frozen, aur Extending the MOP sections incomplete hain. Source ke ek example mein property descriptor defaults ko lekar ek misleading comment bhi hai, jise neeche correct kiya gaya hai.

1. Object ke andar sirf values nahi, behavior bhi hota hai

Normally object ko hum is tarah dekhte hain:

```
1  const user = {  
2      name: "Ruth",  
3      age: 25  
4  };
```

Hume lagta hai:

- name property mein "Ruth" value hai
- age property mein 25 value hai

Lekin JavaScript internally har property ke saath extra information bhi store karta hai:

- Property enumerate hogi ya nahi?
- Property update ki ja sakti hai ya nahi?
- Property delete ya redefine ki ja sakti hai ya nahi?
- Property direct value store karti hai ya getter/setter chalati hai?

In internal rules ko collectively chapter **Metaobject Protocol**, ya **MOP**, keh raha hai.

2. Metaobject Protocol, MOP

Metaobject Protocol ko simply aise samjho:

MOP rules ka ek system hai jo decide karta hai ki object operations kaise behave karenge.

For example, jab hum likhte hain:

```
1 user.name
```

To JavaScript ko decide karna hota hai:

1. Kya name property directly user par present hai?
2. Agar present hai, to kya ye normal value property hai ya getter?
3. Agar present nahi hai, to kya prototype chain mein milegi?
4. Agar getter hai, to kaunsa function execute hoga?

Isi tarah:

```
1 user.name = "Alex";
```

JavaScript check karta hai:

1. Kya property writable hai?
2. Kya uska setter defined hai?
3. Kya object extensible hai?
4. Kya property inherited hai?
5. Strict mode mein failure par error throw karna hai ya nahi?

Ye sab object ke underlying behavior ka part hai.

3. Property Descriptors

Har object property ke saath ek internal description hoti hai, jise **property descriptor** kehte hain.

Example:

```
1 const myObj = {  
2   favoriteNumber: 42,  
3   isDeveloper: true,  
4   firstName: "Kyle"  
5 };
```

favoriteNumber sirf 42 store nahi kar rahi. Uske saath kuch attributes bhi attached hain.

Hum descriptor dekh sakte hain:

```
1 Object.getOwnPropertyDescriptor(  
2   myObj,  
3   "favoriteNumber"  
4 );
```

Result:

```
1 {
```

```
2     value: 42,  
3     enumerable: true,  
4     writable: true,  
5     configurable: true  
6 }
```

Iska meaning:

- value: 42: property ki current value
- enumerable: true: property enumeration mein dikhegi
- writable: true: value assignment se update ho sakti hai
- configurable: true: descriptor change ya property delete ki ja sakti hai

getOwnPropertyDescriptor mein "Own" ka meaning

Method ka naam hai:

```
1 Object.getOwnPropertyDescriptor()
```

Yahan Own ka matlab hai ki method sirf object ki **directly owned property** check karega.

Prototype se inherited property ka descriptor is object par nahi milega.

```
1 const obj = {};  
2  
3 Object.getOwnPropertyDescriptor(obj, "toString");  
4 // undefined
```

obj.toString accessible ho sakta hai, lekin toString directly obj ki own property nahi hai. Wo prototype chain se aa raha hai.

4. Object.defineProperty()

Hum manually kisi property ka descriptor define kar sakte hain:

```
1 const anotherObj = {};  
2  
3 Object.defineProperty(anotherObj, "fave", {  
4     value: 42,  
5     enumerable: true,  
6     writable: true,  
7     configurable: true  
8 });  
9  
10 console.log(anotherObj.fave);
```

Is method se property ki value ke saath uska exact behavior bhi control hota hai.

Chapter ke example mein important correction

Provided chapter mein ye comments diye gaye hain:

```
1 enumerable: true,      // default if omitted
2 writable: true,       // default if omitted
3 configurable: true    // default if omitted
```

Ye comments **incorrect ya misleading** hain.

Object.defineProperty() use karte samay agar ye attributes omit kar diye jayen, to inki default value false hoti hai, true nahi.

For example:

```
1 const obj = {};
2
3 Object.defineProperty(obj, "x", {
4     value: 10
5 });
```

Conceptually descriptor banega:

```
1 {
2     value: 10,
3     writable: false,
4     enumerable: false,
5     configurable: false
6 }
```

Isliye ye property:

- normal assignment se update nahi hogi
- Object.keys() mein nahi dikhegi
- delete ya redefine nahi ki ja sakegi

Object literal se banayi properties ke attributes generally true hote hain:

```
1 const obj = {
2     x: 10
3 };
```

Lekin Object.defineProperty() mein omitted attributes false hote hain.

Ye difference bahut important hai.

5. Existing property ko redefine karna

Agar property:

```
1 configurable: true
```

hai, to generally uska descriptor dobara define kiya ja sakta hai.

```
1 const obj = {};  
2  
3 Object.defineProperty(obj, "score", {  
4     value: 10,  
5     writable: true,  
6     enumerable: true,  
7     configurable: true  
8 });  
9  
10 Object.defineProperty(obj, "score", {  
11     value: 20,  
12     writable: false  
13 });
```

Ab:

```
1 obj.score;  
2 // 20
```

Lekin property non-configurable ho:

```
1 configurable: false
```

to uski definition lock ho jati hai.

```
1 Object.defineProperty(obj, "id", {  
2     value: 101,  
3     configurable: false  
4 });
```

Ab use freely redefine nahi kar sakte:

```
1 Object.defineProperty(obj, "id", {  
2     enumerable: true  
3 });  
4  
5 // TypeError
```

Non-configurable ka matlab completely unchangeable nahi

Agar descriptor hai:

```
1 {  
2   value: 10,  
3   writable: true,  
4   configurable: false  
5 }
```

to direct assignment abhi bhi possible hai:

```
1 obj.someProperty = 20;
```

Kyunki writable: true hai.

Lekin property ke descriptor ko generally redefine nahi kiya ja sakta.

6. Copying properties descriptors ko preserve nahi karti

Chapter ki warning ka main point ye hai ki common copying operations:

```
1 const copy = { ...source };
```

ya:

```
1 Object.assign(copy, source);
```

original property descriptor ko exact copy nahi karte.

Ye operations approximately ye process follow karte hain:

1. Source se property ki value read karo
2. Target par normal assignment/create operation karo

For example:

```
1 const source = {};  
2  
3 Object.defineProperty(source, "secret", {  
4   value: 42,  
5   enumerable: true,  
6   writable: false,  
7   configurable: false  
8 });  
9  
10 const copy = {  
11   ...source  
12 };
```

copy.secret ki value 42 hogi, lekin zaroori nahi ki:

- writable: false

- configurable: false

bhi preserve hon.

Original property ka special behavior lose ho sakta hai.

Getter ho to copying ke time getter bhi execute ho sakta hai, aur target mein us getter ka returned result normal value ke roop mein store ho sakta hai.

7. Object.defineProperty()

Multiple properties ko ek saath descriptors ke through define karne ke liye:

```
1  const user = {};  
2  
3  Object.defineProperty(user, {  
4      name: {  
5          value: "Kyle",  
6          writable: true,  
7          enumerable: true,  
8          configurable: true  
9      },  
10  
11     age: {  
12         value: 42,  
13         writable: false,  
14         enumerable: true,  
15         configurable: true  
16     }  
17 });
```

Ye tab useful ho sakta hai jab multiple properties par precise restrictions lagani hon.

Normal application code mein object literal zyada readable hota hai:

```
1  const user = {  
2      name: "Kyle",  
3      age: 42  
4  };
```

Isliye Object.defineProperty() comparatively kam dikhta hai.

8. Do types ki properties

Property descriptors ke point of view se properties broadly do categories mein aa sakti hain:

1. **Data property**
 2. **Accessor property**
-

8.1 Data property

Data property actual value store karti hai.

```
1 {  
2   value: 42,  
3   writable: true,  
4   enumerable: true,  
5   configurable: true  
6 }
```

Example:

```
1 const obj = {  
2   score: 42  
3 };
```

score ek normal data property hai.

8.2 Accessor property

Accessor property directly fixed value store karne ke bajay getter aur setter functions define karti hai.

Descriptor approximately:

```
1 {  
2   get() {  
3     // value return karo  
4   },  
5  
6   set(value) {  
7     // assignment handle karo  
8   },  
9  
10  enumerable: true,  
11  configurable: true
```

```
12 }
```

Accessor descriptor mein normally:

- get
- set

hote hain.

Data descriptor mein normally:

- value
- writable

hote hain.

Ek hi descriptor ko proper data property aur accessor property dono nahi banana chahiye.

9. Getter aur setter kaise work karte hain?

Example:

```
1  const anotherObj = {};  
2  
3  Object.defineProperty(anotherObj, "fave", {  
4    get() {  
5      console.log("Getting 'fave' value!");  
6      return 123;  
7    },  
8  
9    set(v) {  
10     console.log(`Ignoring ${v} assignment.`);  
11   }  
12 });
```

Jab hum likhenge:

```
1  anotherObj.fave;
```

Dikhne mein ye normal property access hai. Lekin internally get() execute hoga:

```
1  Getting 'fave' value!  
2  123
```

Aur jab likhenge:

```
1  anotherObj.fave = 42;
```

To value directly store nahi hogi. set(v) call hoga, jahan v ki value 42 hogi:

```
1 Ignoring 42 assignment.
```

Getter fir bhi 123 return karega:

```
1 anotherObj.fave;  
2 // 123
```

Ek practical getter/setter example

```
1  const person = {  
2      firstName: "Ruth",  
3      lastName: "Braun",  
4  
5      get fullName() {  
6          return `${this.firstName} ${this.lastName}`;  
7      },  
8  
9      set fullName(value) {  
10         const parts = value.split(" ");  
11  
12         this.firstName = parts[0];  
13         this.lastName = parts[1];  
14     }  
15 };
```

Property ki tarah read:

```
1 person.fullName;  
2 // "Ruth Braun"
```

Property ki tarah assign:

```
1 person.fullName = "Alex Smith";
```

Lekin internally setter execute hua.

Ab:

```
1 person.firstName;  
2 // "Alex"  
3  
4 person.lastName;  
5 // "Smith"
```

10. enumerable

enumerable control karta hai ki property common enumeration aur copying operations mein include hogi ya nahi.

Example:

```
1  const user = {  
2    name: "Kyle"  
3  };  
4  
5  Object.defineProperty(user, "internalId", {  
6    value: 123,  
7    enumerable: false  
8  });
```

Direct access possible hai:

```
1  user.internalId;  
2  // 123
```

Lekin:

```
1  Object.keys(user);  
2  // ["name"]
```

internalId output mein nahi aayegi.

Chapter ke according non-enumerable property generally in operations mein skip ho sakti hai:

- Object.keys()
- Object.entries()
- for...in
- Object spread
- Object.assign()

Example:

```
1  const copiedUser = {  
2    ...user  
3  };  
4  
5  console.log(copiedUser);  
6  // { name: "Kyle" }
```

internalId enumerable nahi thi, isliye spread copy mein include nahi hui.

Non-enumerable ka matlab private nahi

Ye important distinction hai:

```
1 user.internalId;  
2 // 123
```

Property abhi bhi directly accessible hai.

enumerable: false property ko:

- private nahi banata
- secure nahi banata
- inaccessible nahi banata

Bas kuch enumeration operations se hide karta hai.

11. writable

writable decide karta hai ki data property ko = se new value assign ki ja sakti hai ya nahi.

```
1 const user = {};  
2  
3 Object.defineProperty(user, "id", {  
4     value: 101,  
5     writable: false,  
6     enumerable: true,  
7     configurable: true  
8 });
```

Ab:

```
1 user.id = 202;
```

Property update nahi hogi.

Non-strict mode mein assignment silently fail ho sakti hai. Strict mode mein error mil sakta hai:

```
1 "use strict";  
2  
3 user.id = 202;  
4 // TypeError
```

writable: false aur configurable: true

Agar property writable nahi hai, lekin configurable hai:

```
1 {  
2     writable: false,  
3     configurable: true  
4 }
```

to direct assignment fail hogi:

```
1 user.id = 202;
```

Lekin `Object.defineProperty()` ke through value redefine ki ja sakti hai:

```
1 Object.defineProperty(user, "id", {  
2     value: 202  
3 });
```

Kyunki `configurable: true` property descriptor ko change karne ki permission de raha hai.

12. configurable

`configurable` decide karta hai ki property ki overall definition kitni change ki ja sakti hai.

Agar:

```
1 configurable: true
```

hai, to generally:

- descriptor redefine kiya ja sakta hai
- property delete ki ja sakti hai
- data property ko accessor property mein convert kiya ja sakta hai
- accessor ko data property mein convert kiya ja sakta hai

Agar:

```
1 configurable: false
```

hai, to property definition largely lock ho jati hai.

Example:

```
1 const obj = {};  
2  
3 Object.defineProperty(obj, "id", {  
4     value: 101,  
5     writable: true,  
6     enumerable: true,  
7     configurable: false  
8 });
```

Value update ho sakti hai:

```
1 obj.id = 202;
```

Kyunki writable: true hai.

Lekin property delete nahi hogi:

```
1 delete obj.id;
```

Aur descriptor freely change nahi kar sakte.

13. Object Sub-types

JavaScript mein kuch values technically objects hoti hain, lekin unka special behavior bhi hota hai.

Chapter do common object sub-types discuss karta hai:

1. Arrays
2. Functions

Matlab array aur function plain object nahi hain, lekin object capabilities bhi rakhte hain.

14. Arrays are objects

Array numerically indexed values ke liye designed hota hai:

```
1 const myList = [23, 42, 109];
```

Access:

```
1 myList[0];  
2 // 23  
3  
4 myList[1];  
5 // 42
```

Arrays zero-indexed hote hain. First item index 0 par hota hai.

Arrays mein mixed values

JavaScript arrays mein different types ki values store ho sakti hain:

```
1 const values = [  
2     42,  
3     "hello",
```

```
4     true,  
5     { name: "Kyle" },  
6     [1, 2],  
7     function help() {}  
8 ];
```

JavaScript is mixture ko allow karta hai.

15. Array indexes internally properties hain

Array technically object hai. Isliye:

```
1 myList[2]
```

ek property access hai.

String form bhi kaam kar sakti hai:

```
1 myList["2"];  
2 // 109
```

Kyunki "2" valid integer index jaisa represent hota hai.

Lekin readable aur intended form ye hai:

```
1 myList[2];
```

Na ki:

```
1 myList["2"];
```

16. Array par named properties

Technically allowed:

```
1 const list = [10, 20];  
2  
3 list.favoriteNumber = 42;
```

Ab:

```
1 list.favoriteNumber;  
2 // 42
```

Lekin ye generally avoid karna chahiye. Array numerically indexed collection ke liye hai.

Agar named information rakhni hai, separate object better hai:

```
1 const data = {
```

```
2     favoriteNumber: 42,  
3     list: [10, 20]  
4 };
```

Isse data structure clear rehta hai.

17. Array ki length property

Har array par automatically length property hoti hai:

```
1  const myList = [23, 42, 109];  
2  
3  myList.length;  
4  // 3
```

Value push karne par length update ho jati hai:

```
1  myList.push("Hello");  
2  
3  myList.length;  
4  // 4
```

length getter nahi hai

Chapter ka important point hai ki developers kabhi-kabhi sochte hain:

```
1  myList.length
```

har access par array ko count karta hoga.

Lekin chapter ke mutabik ye getter ki tarah on-the-fly complete array length calculate nahi karta.

Purane loops mein log ye optimization karte the:

```
1  for (  
2      let i = 0, len = myList.length;  
3      i < len;  
4      i++)  
5  ) {  
6      // ...  
7  }
```

Is assumption se ki length baar-baar access karna expensive hai.

Chapter ka recommendation hai ki modern JavaScript engines ko apna kaam karne do:

```
1  for (let i = 0; i < myList.length; i++) {
```

```
2    // ...
3 }
```

Unnecessary manual optimization readability ko reduce kar sakti hai.

18. Empty Slots

Array ka ek problematic behavior hai: **empty slots**.

Example:

```
1  const myList = [23, 42, 109];
2
3  myList[14] = "Hello";
```

Pehle indexes the:

```
1  0, 1, 2
```

Directly index 14 assign kar diya. JavaScript indexes 3 se 13 ke beech actual undefined values automatically fill nahi karta. Wo empty slots chhod deta hai.

Array conceptually:

```
1  [
2      23,
3      42,
4      109,
5      empty × 11,
6      "Hello"
7  ]
```

Length:

```
1  myList.length;
2  // 15
```

Kyunki highest index 14 hai, aur array length highest index se one greater hoti hai.

Empty slot read karne par confusion

```
1  myList[9];
2  // undefined
```

Result undefined hai, lekin iska matlab ye zaroori nahi ki index 9 par actual property present hai jiska value undefined hai.

Compare:

```
1  const a = [];
```

```
2 a[0] = undefined;
```

Yahan index 0 actual property hai:

```
1 Object.hasOwn(a, 0);  
2 // true
```

Lekin empty slot mein:

```
1 const b = new Array(1);  
2  
3 Object.hasOwn(b, 0);  
4 // false
```

Dono ko read karne par undefined mil sakta hai, lekin internal structure different hai.

Empty slots harmful kyun hain?

Kuch array methods empty slots skip kar dete hain.

Example:

```
1 const values = [];  
2  
3 values[2] = 10;  
4  
5 values.map(function double(value) {  
6     return value * 2;  
7 });
```

Indexes 0 aur 1 empty hain. map() callback un slots ke liye necessarily execute nahi karega.

Isliye chapter ki strong recommendation:

Intentionally empty slots create mat karo.

Instead:

```
1 const values = [undefined, undefined, 10];
```

Ya actual application requirement ke according array ko continuously build karo:

```
1 const values = [];  
2  
3 values.push(10);  
4 values.push(20);  
5 values.push(30);
```

19. Functions are also objects

JavaScript function executable bhi hota hai aur object bhi.

```
1 function help() {  
2     console.log("Help");  
3 }
```

Function call ho sakta hai:

```
1 help();
```

Aur technically us par properties bhi ho sakti hain:

```
1 help.description = "Provides help";  
2  
3 console.log(help.description);  
4 // "Provides help"
```

Lekin chapter baad mein explain karta hai ki custom properties function par attach karna generally avoid karna chahiye.

20. Function ki name property

```
1 function help(opt1, opt2, ...remainingOpts) {  
2     // ...  
3 }  
4  
5 help.name;  
6 // "help"
```

name function ka inferred ya declared name batati hai.

21. Function ki length property

Function ki length array length nahi hoti.

```
1 function help(opt1, opt2, ...remainingOpts) {  
2     // ...  
3 }  
4  
5 help.length;  
6 // 2
```

Function length explicitly declared parameters ki count batati hai, lekin count:

- first default parameter se pehle tak
- rest parameter ko include kiye bina

Example:

```
1 function example(a, b, c) {}
2
3 example.length;
4 // 3
```

Default parameter:

```
1 function example(a, b = 10, c) {}
2
3 example.length;
4 // 1
```

Count b par stop ho gayi, kyunki b ke paas default value hai.

Rest parameter:

```
1 function example(a, b, ...remaining) {}
2
3 example.length;
4 // 2
```

remaining count nahi hua.

22. Function-object properties avoid kyun karein?

Technically possible:

```
1 function help() {}
2
3 help.extraInfo = "Important information";
```

Chapter recommend karta hai ki function ko executable behavior ke liye use karo aur associated metadata separate Map ya WeakMap mein rakho.

```
1 const extraInfo = new Map();
2
3 extraInfo.set(
4   help,
5   "This is some important information"
6 );
```

Retrieve:

```
1 extraInfo.get(help);
2 // "This is some important information"
```

Is design ka benefit:

- Function aur metadata separate rehte hain
- Built-in function properties ke saath conflicts kam hote hain
- Code ki responsibility clear rehti hai
- Function object unnecessarily mutate nahi hota

Map mein function khud key ban sakta hai, kyunki objects aur functions valid Map keys hain.

23. Object-wide characteristics

Property descriptor ek individual property ka behavior control karta hai.

Lekin kuch controls entire object par apply ho sakte hain:

1. **Extensible**
2. **Sealed**
3. **Frozen**

Chapter mein Extensible explain hua hai, lekin Sealed aur Frozen sections TODO hain.

24. Extensible objects

By default objects extensible hote hain.

Meaning:

Existing object par new properties add ki ja sakti hain.

```
1 const myObj = {
2     favoriteNumber: 42
3 };
4
5 myObj.firstName = "Kyle";
```

firstName successfully add ho gayi.

Extensibility disable karne ke liye:

```
1 Object.preventExtensions(myObj);
```

Ab new property add nahi hogi:

```
1 myObj.nicknames = ["getify", "ydkjs"];
```

Lekin existing property update ho sakti hai:

```
1 myObj.favoriteNumber = 123;
```

Kyunki `preventExtensions()` ka meaning ye nahi hai ki object completely read-only ho gaya.

Ye bas new properties ko rokta hai.

Check karna ho object extensible hai ya nahi

Chapter ke main text mein ye method show nahi hua, lekin conceptually check is form mein hota hai:

```
1 Object.isExtensible(myObj);
```

Result object ki current extensibility represent karega.

Strict mode difference

Non-strict mode mein forbidden property add karna silently fail ho sakta hai.

```
1 const obj = {};  
2  
3 Object.preventExtensions(obj);  
4  
5 obj.newProperty = 42;
```

Strict mode mein exception ho sakta hai:

```
1 "use strict";  
2  
3 const obj = {};  
4  
5 Object.preventExtensions(obj);  
6  
7 obj.newProperty = 42;  
8 // TypeError
```

25. Sealed aur Frozen

Provided material mein dono sections TODO hain, isliye chapter ne inhe detail mein explain nahi kiya.

Sirf overall relationship ko chapter ki direction ke according aise understand karo:

- **Extensibility** new properties add hone ko control karti hai
- **Sealing** object structure ko stronger level par lock karne ka concept hai

- **Freezing** values aur structure ko aur stronger level par restrict karne ka concept hai

Lekin source mein detailed explanation nahi di gayi, isliye in sections ko completed chapter content nahi maanna chahiye.

26. `[[Prototype]]` Chain

Ye chapter ka sabse important aur confusing section hai.

Har normal object ke paas ek internal linkage ho sakta hai:

```
1  [[Prototype]]
```

Ye linkage kisi doosre object ko point karta hai.

Important:

```
1  [[Prototype]]
```

aur:

```
1  prototype
```

same cheez nahi hain.

Dono ke names similar hain, isi wajah se confusion hoti hai.

27. Prototype chain ka basic idea

Example:

```
1  const myObj = {  
2          favoriteNumber: 42  
3  };
```

`myObj` ke code mein hume sirf `favoriteNumber` dikh raha hai.

Lekin JavaScript normally is object ka internal `[[Prototype]]` link built-in object:

```
1  Object.prototype
```

ki taraf set kar deta hai.

Conceptual structure:

```
1  myObj  
2  |  
3  | [[Prototype]]
```

```
4    v
5  Object.prototype
6    |
7    | [[Prototype]]
8    v
9  null
```

null ka meaning chain ka end hai.

28. Property lookup delegation

Jab hum likhte hain:

```
1  myObj.favoriteNumber
```

JavaScript ko property directly myObj par mil jati hai.

Lekin jab hum likhte hain:

```
1  myObj.toString()
```

toString directly myObj par defined nahi hai.

JavaScript approximately ye steps follow karta hai:

1. myObj par toString search karo
2. Nahi mili
3. myObj ke [[Prototype]] par jao
4. Object.prototype par toString search karo
5. Method mil gayi
6. Use access/call karo

Ye lookup dusre object ko **delegate** ho gaya.

Chapter author “inheritance” se zyada “delegation” word prefer karta hai.

29. Inheritance ya delegation?

Common terminology:

| myObj ne Object.prototype se toString() inherit kiya.

Chapter author's preferred conceptual explanation:

| myObj par property nahi mili, isliye lookup Object.prototype ko delegate ho gaya.

Difference mental model ka hai.

“Inheritance” sun kar lag sakta hai ki method ki copy myObj ke andar aa gayi hai. Lekin actually method copy nahi hoti. Lookup chain par aage continue hota hai.

Isliye delegation technically zyada clear mental model deta hai.

30. Object.prototype se commonly available properties

Chapter ne kuch properties list ki hain:

```
1 constructor
2 __proto__
3 toString()
4 valueOf()
5 hasOwnProperty()
6 isPrototypeOf()
```

Normal objects inhe prototype chain ke through access kar sakte hain.

Example:

```
1 const myObj = {
2     favoriteNumber: 42
3 };
4
5 myObj.toString();
6 // "[object Object]"
```

Aur:

```
1 myObj.hasOwnProperty("favoriteNumber");
2 // true
```

31. hasOwnProperty() versus Object.hasOwnProperty()

Old/common form:

```
1 myObj.hasOwnProperty("favoriteNumber");
```

Modern static form:

```
1 Object.hasOwnProperty(myObj, "favoriteNumber");
```

Chapter modern form ko prefer karta hai:

```
1 Object.hasOwn(myObj, "favoriteNumber");  
2 // true
```

Iska meaning:

Kya favoriteNumber directly myObj ki own property hai?

Ye inherited properties ko own nahi maanta:

```
1 Object.hasOwn(myObj, "toString");  
2 // false
```

Bhale hi:

```
1 myObj.toString
```

accessible hai.

Object.hasOwn() more robust kyun hai?

Kuch objects ke paas hasOwnProperty() available nahi ho sakta:

```
1 const dictionary = Object.create(null);
```

Yahan:

```
1 dictionary.hasOwnProperty  
2 // undefined
```

Ya object apni custom property bana kar method ko shadow kar sakta hai:

```
1 const obj = {  
2   hasOwnProperty: "not a function"  
3 };
```

Is case mein:

```
1 obj.hasOwnProperty("x");
```

fail ho jayega.

Lekin:

```
1 Object.hasOwn(obj, "x");
```

safe aur direct form hai.

32. Different [[Prototype]] ke saath object create karna

Default object normally Object.prototype se linked hota hai.

Lekin hum different object se linkage create kar sakte hain:

```
1 const differentObj = {
```

```
2     greet() {
3         return "Hello";
4     }
5 };
6
7 const myObj = Object.create(differentObj);
```

Ab myObj khud empty hai, lekin:

```
1 myObj.greet();
2 // "Hello"
```

Lookup process:

```
1 myObj
2   |
3   | [[Prototype]]
4   v
5 differentObj
6   |
7   | [[Prototype]]
8   v
9 Object.prototype
10  |
11  v
12 null
```

greet directly myObj ki own property nahi hai:

```
1 Object.hasOwn(myObj, "greet");
2 // false
```

Lekin chain ke through accessible hai:

```
1 "greet" in myObj;
2 // true
```

Yahan Object.hasOwn() aur in ka difference clearly dikhta hai.

33. Object.create() ka second argument

Object.create() ka optional second argument properties define kar sakta hai:

```
1 const myObj = Object.create(differentObj, {
2     name: {
3         value: "Kyle",
```

```
4     writable: true,  
5     enumerable: true,  
6     configurable: true  
7   }  
8 });
```

Ye simultaneously:

1. Object ka prototype set karta hai
2. Initial properties descriptors ke through create karta hai

Lekin syntax normal object literal se verbose hai. Isliye practical code mein comparatively kam use hota hai.

34. Object literal mein `__proto__`

Chapter alternate syntax show karta hai:

```
1  const myObj = {  
2    __proto__: differentObj,  
3  
4    name: "Kyle"  
5  };
```

Isse created object ka internal `[[Prototype]]` differentObj ho sakta hai.

Lekin chapter ise **less preferable** maanta hai.

Preferred form:

```
1  const myObj = Object.create(differentObj);
```

`__proto__` historically unusual aur environment-dependent feature raha hai. Isliye production design mein explicit standard APIs ko prefer karna better hota hai.

35. Empty `[[Prototype]]` object

Normal object:

```
1  const obj = {};
```

usually `Object.prototype` se linked hota hai.

Prototype-less object:

```
1  const emptyObj = Object.create(null);
```

Iska internal chain:

```
1 emptyObj
2   |
3   | [[Prototype]]
4   v
5  null
```

Ab:

```
1 emptyObj.toString;
2 // undefined
```

Kyunki Object.prototype chain mein hai hi nahi.

36. Dictionary objects

Prototype-less objects ko kabhi-kabhi **dictionary objects** kaha jata hai.

Example:

```
1 const dictionary = Object.create(null);
2
3 dictionary.apple = 10;
4 dictionary.toString = 20;
5 dictionary.constructor = 30;
```

Normal object mein toString aur constructor names inherited properties ke saath collide kar sakte hain.

Lekin null-prototype object mein inherited properties nahi hoti. Isliye arbitrary string keys rakhna comparatively safer ho sakta hai.

Example:

```
1 "toString" in dictionary;
2 // true
```

Yahan result sirf isliye true hai kyunki humne toString directly define ki hai, na ki Object.prototype se inheritance ke karan.

Limitation

Prototype-less object par common inherited utilities nahi hongi:

```
1 dictionary.toString();
2 // TypeError
```

Aur:

```
1 dictionary.hasOwnProperty("apple");
2 // TypeError
```

Isliye own property check ke liye:

```
1 Object.hasOwnProperty(dictionary, "apple");
2 // true
```

use karna appropriate hai.

37. in versus Object.hasOwnProperty()

Difference bahut important hai.

Object.hasOwnProperty()

Sirf direct own property check karta hai:

```
1 Object.hasOwnProperty(obj, "propertyName");
```

in

Own plus prototype chain dono check karta hai:

```
1 "propertyName" in obj;
```

Example:

```
1 const parent = {
2     inheritedValue: 42
3 };
4
5 const child = Object.create(parent);
6
7 child.ownValue = 100;
```

Results:

```
1 Object.hasOwnProperty(child, "ownValue");
2 // true
3
4 Object.hasOwnProperty(child, "inheritedValue");
5 // false
```

Lekin:

```
1 "ownValue" in child;
2 // true
3
```

```
4 "inheritedValue" in child;  
5 // true
```

38. [[Prototype]] versus prototype

Ye distinction carefully samjho.

[[Prototype]]

- Har object ka internal linkage ho sakta hai
- Property lookup delegation mein use hota hai
- Ye conceptual/internal slot hai
- Kisi aur object ko point karta hai

Example conceptually:

```
1 myObj.[[Prototype]] -> Object.prototype
```

prototype

- Ye functions par commonly present ek public property hai
- Ye khud ek object ko refer karti hai
- Jab function ko new ke saath call kiya jata hai, newly created object ka [[Prototype]] generally us function ki .prototype property ke object ko point karta hai

Example:

```
1 function Person() {}
```

Function par:

```
1 Person.prototype
```

ek object hai.

Agar:

```
1 const person = new Person();
```

to conceptual relationship:

```
1 person.[[Prototype]] -> Person.prototype
```

Important:

```
1 Person.prototype
```

function Person ka apna prototype-chain linkage nahi batata.

Ye batata hai ki new Person() se banne wale objects kis object se link honge.

39. Object literal aur new Object()

Chapter ke according:

```
1 const myObj = {};
```

conceptually largely similar hai:

```
1 const myObj = new Object();
```

Dono cases mein newly created object usually:

```
1 Object.prototype
```

se `[[Prototype]]`-linked hota hai.

Conceptually:

```
1 myObj.[[Prototype]] -> Object.prototype
```

Yahan:

- Object ek function hai
- Object.prototype us function ki public prototype property par available object hai
- myObj ka internal `[[Prototype]]` us object ko point karta hai

Isi overlapping naming ki wajah se topic confusing lagta hai.

40. Functions ka apna `[[Prototype]]`

Functions khud bhi objects hain.

Example:

```
1 function help() {}
```

help function-object ka apna internal `[[Prototype]]` generally:

```
1 Function.prototype
```

ko point karta hai.

Conceptually:

```
1 help
2   |
3   | [[Prototype]]
4   v
5   Function.prototype
6   |
7   | [[Prototype]]
```

```
8    v
9    Object.prototype
10   |
11   v
12  null
```

Isi chain ki wajah se functions ko methods milti hain:

```
1  help.call(...)
2  help.apply(...)
3  help.bind(...)
4  help.toString()
```

call, apply, aur bind directly help par copy nahi hoti. Ye prototype chain ke through accessible hoti hain.

41. Object.prototype bhi chain ka part hai

Function.prototype khud ek object hai. Isliye wo bhi ultimately Object.prototype ki capabilities access kar sakta hai.

Function chain ko simplified form mein:

```
1  help
2    -> Function.prototype
3    -> Object.prototype
4    -> null
```

Normal object chain:

```
1  myObj
2    -> Object.prototype
3    -> null
```

Custom prototype chain:

```
1  child
2    -> parent
3    -> Object.prototype
4    -> null
```

Null-prototype dictionary:

```
1  dictionary
2    -> null
```

42. Property lookup ka complete mental model

Jab expression hai:

```
1 obj.someProperty
```

to simplified lookup aise samjho:

1. JavaScript obj ki own properties check karta hai
2. Mil gayi to us descriptor ke according value/getter handle karta hai
3. Nahi mili to obj.[[Prototype]] ke target object par search karta hai
4. Wahan bhi nahi mili to us target ke [[Prototype]] par jata hai
5. Ye process chain mein continue hota hai
6. null tak property nahi mili to result undefined

Conceptual pseudo-code:

```
1 function lookup(obj, key) {  
2     if (Object.hasOwn(obj, key)) {  
3         return obj[key];  
4     }  
5  
6     const parent = getInternalPrototype(obj);  
7  
8     if (parent === null) {  
9         return undefined;  
10    }  
11  
12    return lookup(parent, key);  
13 }
```

Real JavaScript engine implementation exactly ye code nahi chalati, lekin conceptual understanding ke liye model useful hai.

43. Prototype method mein this

Chapter ke conclusion mein ek important line hai:

Delegated method ke execution ka context initial object se target method tak this ke through share hota hai.

Example:

```
1 const parent = {  
2     describe() {
```

```
3         return `My value is ${this.value}`;  
4     }  
5 };  
6  
7 const child = Object.create(parent);  
8  
9 child.value = 42;
```

Call:

```
1 child.describe();  
2 // "My value is 42"
```

describe method parent par defined hai, lekin call hua:

```
1 child.describe()
```

Isliye method ke andar:

```
1 this
```

child ko refer karta hai.

Method lookup parent par delegate hua, lekin call-site context child ka raha.

Ye delegation pattern ka powerful part hai.

44. Object behavior ka combined example

```
1 const behavior = {  
2     describe() {  
3         return `${this.name}: ${this.score}`;  
4     }  
5 };  
6  
7 const player = Object.create(behavior);  
8  
9 Object.defineProperty(player, {  
10     name: {  
11         value: "Ruth",  
12         writable: true,  
13         enumerable: true,  
14         configurable: true  
15     },  
16
```

```
17     score: {
18         value: 100,
19         writable: false,
20         enumerable: true,
21         configurable: false
22     },
23
24     secretId: {
25         value: 999,
26         writable: false,
27         enumerable: false,
28         configurable: false
29     }
30 });
```

Ab analyze karte hain.

Own properties

```
1 Object.hasOwn(player, "name");
2 // true

1 Object.hasOwn(player, "score");
2 // true

1 Object.hasOwn(player, "describe");
2 // false
```

describe prototype chain se aa raha hai.

Enumeration

```
1 Object.keys(player);
2 // ["name", "score"]
```

secretId enumerable nahi hai, isliye output mein nahi aayi.

Direct access

```
1 player.secretId;
2 // 999
```

Non-enumerable hone ke baad bhi direct access possible hai.

Non-writable property

```
1 player.score = 200;
```

score update nahi hogi.

Delegated method

```
1 player.describe();  
2 // "Ruth: 100"
```

Method behavior par hai, lekin this player hai.

Prototype-aware lookup

```
1 "describe" in player;  
2 // true
```

Kyunki in prototype chain check karta hai.

45. Chapter ke major takeaways

1. Object property sirf key-value pair nahi

Har property ka descriptor hota hai jo uska behavior define karta hai.

2. Descriptor ke important attributes

Data property:

```
1 {  
2   value,  
3   writable,  
4   enumerable,  
5   configurable  
6 }
```

Accessor property:

```
1 {  
2   get,  
3   set,  
4   enumerable,  
5   configurable  
6 }
```

3. enumerable visibility nahi, enumeration control hai

Non-enumerable property direct access se fir bhi available ho sakti hai.

4. writable assignment control karta hai

1 obj.prop = value

allowed hai ya nahi, ye writable decide karta hai.

5. configurable property definition ko lock karta hai

Once configurable: false, property ko freely redefine ya delete nahi kiya ja sakta.

6. Arrays objects hain

Lekin wo numeric/indexed collections ke liye specialized hain.

Named properties array par attach karna technically possible hai, lekin avoid karna chahiye.

7. Empty slots avoid karo

Empty slot aur actual undefined value same cheez nahi hain. Kuch array methods empty slots skip kar dete hain.

8. Functions bhi objects hain

Functions ke paas name, length, aur inherited methods jaise call, apply, bind hote hain.

Custom metadata directly function par attach karne ke bajay Map ya WeakMap useful ho sakta hai.

9. Object.preventExtensions() sirf new properties rokta hai

Existing writable properties abhi bhi update ho sakti hain.

10. Prototype chain property lookup delegation karti hai

Agar property object par nahi mili, JavaScript uske [[Prototype]] object par search continue karta hai.

11. `[[Prototype]]` aur `.prototype` different hain

```
1  [[Prototype]]
```

object ka internal linkage hai.

```
1  SomeFunction.prototype
```

function ki public property hai, jo usually `new SomeFunction()` se banne wale objects ke linkage target ke roop mein use hoti hai.

12. `Object.hasOwn()` prefer karo

```
1  Object.hasOwn(obj, "property")
```

sirf direct owned property check karta hai aur instance `hasOwnProperty()` se zyada robust hai.

Final mental model

JavaScript object ko is tarah imagine karo:

```
1  Object
2  |— Own properties
3  |   |— Property value ya getter/setter
4  |   |— enumerable
5  |   |— writable
6  |   |— configurable
7  |
8  |— Object-level state
9  |   |— extensible
10 |   |— sealed
11 |   |— frozen
12 |
13 |— Internal [[Prototype]] link
14 |   |— Another object
15 |       |— Another object
16 |           |— Object.prototype
17 |               |— null
```

Jab property access hota hai:

```
1  obj.property
```

to JavaScript pehle object ki own property dekhta hai. Agar nahi milti, to `[[Prototype]]` chain par lookup delegate karta hai. Agar property mil jati hai, to uske descriptor ke rules ke according value return hoti hai ya getter execute hota hai.

Explanation approach

Maine chapter ko original order aur concepts ke according explain kiya hai, incomplete TODO sections ko clearly mark kiya hai, aur `Object.defineProperty()` ke omitted descriptor attributes wale misleading comments ko correct kiya hai. Sabse important focus property descriptors, arrays ke empty slots, function objects, extensibility, aur `[[Prototype]]` versus `.prototype` distinction par rakha gaya hai.